Ders 5 Genişletilmiş Notu: Sıralamanın Biçimsel Belirtimi ve OCL (Object Constraint Language)

Ders 5 Genişletilmiş Notu: Sıralamanın Biçimsel Belirtimi ve OCL (Object Constraint Language)

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin planı
- 2. Sıralama problemi: Türkçe tanımdan kesin tanıma
- 3. Girdiye olan duyarlılık
- 4. Döngüsellik (circularity)
- 5. Biçimsel belirtimin parçaları: imza, ön koşul, son koşul
- 6. Sıralama için son koşullar
- 7. Permütasyon
- 8. Aynı belirtim OCL ile
- 9. OCL: UML'in resmî bir parçası
- 10. OCL ile neler tanımlanır?
- 11. OCL sözdizimi ve kısıt tipleri
- 12. Değişmezler (invariant)
- 13. Ön ve son koşullar
- 14. Özellik değerlerinde değişiklik ve @pre
- 15. OCL veri tipleri ve anahtar kelimeleri
- 16. Let ifadesi
- 17. Gezinti, çoğullama ve koleksiyonlar
- 18. Diğer OCL özellikleri
- 19. Kütüphane örneğine dönüş: hangi gereksinim OCL ister?
- 20. Gereksinim 6: ödünç periyodu
- 21. Gereksinim 3: ödünç alınanlar
- 22. Gereksinim 4: çocuklar için sınır
- 23. Gereksinim 5: ödünç alma işlemi
- 24. Türetilen özellik: yaş
- 25. Eksik kalanlar ve OCL'nin amacı
- Örnek Sorular

Ders iki bölümden oluşur. İlk bölümde önceki derste yarım kalan biçimsel belirtim konusu, sıralama probleminin birinci derece mantıkla (FOL) belirtimi üzerinden tamamlanır; slayt destesi BLM5126-05'tir (ekrandan, 00:10:00). İkinci bölümde yeni slayt destesine (BLM5126-06, 41 slayt) geçilir ve OCL (Object Constraint Language) tanıtılır, ardından önceki derslerin kütüphane örneğinin gereksinimleri için OCL kısıtları yazılır (ekrandan, 00:40:00).

1. Dersin planı

Hoca günün konusunun OCL olduğunu söyler, ama önce geçen hafta kaldığı yerden devam eder. Yazılımla ilgili bir problemi ya da tasarımı herkesin aynı şekilde anlayacağı, değişmeyecek bir biçimde ifade etmek gerektiğinde nasıl bir kısıt yazmak gerektiği, bildiğimiz basit

bir problem üzerinden incelenecektir. Önce FOL ile bir ifade kurulacak, sonra aynısı OCL ile yazılacaktır.

Hoca OCL'nin UML'in standart bir parçası olduğunu, çok kişi kullanmasa da biçimsel tanım gereken ortamlarda mutlaka bu kısıt diliyle tanım yapıldığını söyler. OCL doğrudan FOL'e dönüştürülebilir. İnsanların daha rahat okuyup anlayacağı bir sözdizimi sağlaması ekstra bir avantajdır: her şeyi tam matematik ve mantık ifadeleriyle göstermek gerekmez. UML araçlarında da tanımlı olduğu için diyagramlarda kullanılabilir.

2. Sıralama problemi: Türkçe tanımdan kesin tanıma

Örnek, herkesin bildiği sıralama problemidir. Elimizde bir liste ya da dizi vardır ve onu nasıl sıralayacağımıza dair sezgisel ya da deneysel yaklaşımlarımız vardır. Hoca bunu iskambil kâğıtlarını sıralamaya benzetir: elinize bir deste alırsınız, astan papaza kadar 13 kâğıt vardır ve bunları artan sırada dizmeniz gerekir. Bunu elinizde yapabilirsiniz ama bilgisayar nasıl yapılacağını kendiliğinden bilmez; ona sıralamanın ne olduğunu söylemek gerekir.

Problemi sadeleştirmek için gerçel sayılar karıştırılmaz; elimizde tam sayı vektörleri vardır:

- Girdi: X, bir tam sayı vektörü.
- Çıktı: Y, X'in artan sırada sıralanmış hâli.

İlk Türkçe tanım şöyle olabilir: "X tam sayı dizisi verildiğinde Y tam sayı dizisini döndürürüz ve Y'deki her eleman bir sonrakinden daha küçüktür." Güzel bir tanım gibi görünür, ama eşit elemanları kaçırır. Hoca iskambil destesinden örnek verir: hem kupalar hem maçalar bir aradaysa her değerden ikişer tane olabilir. O zaman tanım "Y'deki her eleman bir sonrakinden küçük veya eşittir" olmalıdır. Bir sorun daha vardır: son elemanın sonrası yoktur. Demek ki bu kural son elemandan bir öncekine kadar uygulanabilmelidir.

Hoca bu tür tanımları yaparken ilk düşünülmesi gereken şeyin belirtimin girdiyi tanımlayıp tanımlamadığı olduğunu söyler. "Tam sayı vektörü" denmeseydi, yalnız "x" deyip geçilseydi, elmalar, armutlar, muzlar da girdi olabilir ve FOL ifadesinin bunların hepsini anlamlı biçimde sıralaması beklenirdi. Tam sayı vektörü demek girdi kısıtını tam olarak belirler. Aynısı çıktı için de geçerlidir: çıktı düzgün tanımlanmalı, her konumdaki değerin onu takip edenden küçük veya eşit olması belirtilmelidir. Son eleman, takip eden elemanı olmadığı için özel bir durumdur.

3. Girdiye olan duyarlılık

Yalnız çıktının sıralı olduğunu söylemek yetmez. Slaytta bu "önemli bir özellik" olarak verilir (ekrandan, 00:10:00):

- Çıktılarımızın içeriği, girdilerimizin içeriği ile birebir aynı ve tam sayı olmalıdır.
  - 3, 2, 1 girdi, 4, 5, 6 çıktı diyelim. Çıktı önceki tanımımıza uyuyor mu?
- "Çıktı Y vektöründeki her eleman, girdi X vektöründe de yer almalıdır."
  - Peki ya 3, 2, 1 girdiğinde 1, 1, 1, 1, 2, 3 çıkarsa?
- "Çıktı Y vektöründeki her eleman, girdi X vektöründe aynı sayıda yer almalıdır."
  - Peki ya 3, 2, 1 girdiğinde 1, 2, 3, 4 çıkarsa?
- "Çıktı Y vektöründeki her eleman, girdi X vektöründe aynı sayıda yer almalıdır ve yeni eleman yer almamalıdır."
- Hani kısa ve kesin tanım yapacaktık? Sıralama çok basit bir problem. Ya askerî bir yazılımda güvenlik için program yazıyorsak? Kesin ve net tanım yapabilmeliyiz.

Hoca adımları şöyle anlatır. İlk tanıma 3, 2, 1 vektörü verilsin ve program 4, 5, 6 üretsin. Çıktı sıralıdır, girdi de tam sayı vektörüdür; yani hem girdi hem çıktı tanıma uyar. Ama girdiyle çıktının içeriklerinin aynı olması gerektiği koşulu eklenmezse mantık ifadesi bu girdi ve çıktı için de doğru olur.

İkinci koşul eklenir: çıktıdaki her eleman girdide de yer almalıdır. Bu kez 3, 2, 1 verilir ve program 1, 1, 1, 1, 2, 3 üretir. Her eleman girdide vardır, içerik tam sayıdır, ama sayıları farklıdır. Bu çıktı da koşulu sağlar, fakat girdinin sıralanmış hâli değildir. Demek ki bir de sayı kontrolü gerekir: Y'deki her eleman X'teki elemanlarla aynı sayıda yer almalıdır.

Bu sefer 3, 2, 1 için 1, 2, 3, 4 üretilebilir: 1, 2 ve 3 aynı sayıda yer alır, ama 4 girdide yoktur. Bu yüzden daha net bir ifadeyle "başka eleman ya da yeni eleman yer almamalıdır" da eklenmelidir.

> **Hocanın vurgusu:** Sıralama çok basit bir problemdir ama uç noktaları, girdiye olan duyarlılığı ve bunun çıktıda yaratabileceği değişiklikleri garanti altına almak gerekir. Askerî ya da güvenlikle ilgili bir yazılımda bu tanımların ve tasarım noktalarının çok daha net ve kesin olması, kesin çıktılar üretecek şekilde kısıtlanması mutlaka gerekir.
>
> **Ders sonrası not:** Slayttaki üçüncü cümle Y'deki her eleman için yazıldığından harfiyen okunursa, 1, 2, 3, 4 çıktısındaki 4 (X'te 0 kez, Y'de 1 kez geçer) bu koşulu zaten sağlamaz. Hocanın karşı örneği büyük olasılıkla koşulun yalnız X'teki elemanların sayısına bakan okuması için düşünülmüştür. Dört çıktı Python ile denendi: dördü de sıralıdır, ama 3, 2, 1 girdisinin permütasyonu olan tek çıktı 1, 2, 3'tür.

4. Döngüsellik (circularity)

FOL ya da OCL tanımı yazarken dikkat edilecek noktalardan biri döngüselliktir: bir kavramı tanımlarken kavramın kendisini tanımda kullanmak. Sıralamayı sıralama terimleriyle ifade etmek problemi anlamaya hiçbir şey katmaz. Hoca bunu şöyle örnekler: "Bardak nedir?" sorusuna "bardak öyle bir şeydir ki bardak tipindeki şeylere bardak denir" diye cevap vermek hiçbir şey katmaz. Bu yüzden problem ifadeleri tanımın içinde kullanılmamalıdır.

Tanım bir kez daha gözden geçirilir:

- X tam sayı vektörü verildiğinde Y tam sayı vektörü üretilir.
- Y vektörü sıralı olmalıdır.
- Y vektörünün içeriği X vektörünün içeriğiyle aynı olmalıdır; başka bir deyişle, Y vektöründeki her eleman X'ten gelmelidir.
- Y'deki her elemanın tekrar sayısı X'teki tekrar sayısıyla aynı olmalıdır.

Buradaki "Y vektörü sıralı olmalıdır" ifadesi döngüseldir. Bunun yerine "her eleman sonrakinden küçük veya eşit olmalıdır" gibi net ve döngüsellik içermeyen bir tanım yapılmalıdır.

5. Biçimsel belirtimin parçaları: imza, ön koşul, son koşul

Tanımı FOL ile, yani kesin matematik diliyle modellemeye biçimsel belirtim (formal specification) denir. Üç aşaması vardır.

5.1 İmza (signature)

İmza, bir fonksiyon başlığı gibi düşünülebilir; hoca bunu "adınız, soyadınız" gibi bir şey olarak anlatır. Belirtimin ne aldığını, ne verdiğini ve ne iş yaptığını tanımlar: adı, girdinin adı, çıktının adı ve tipleri. Sıralama için imza, tam sayı vektöründen oluşan bir x girdisi alan, adı `sort` olan ve `y = sort(x)` biçiminde yine bir tam sayı vektörü döndüren bir yapıdır. Tek girdi ve tek çıktı vardır. Bunlara kesin adlar verilmelidir, çünkü ön ve son koşullar tarif edilirken x ve y kullanılacaktır; mantık olarak Java'daki değişkenler gibi düşünülebilir.

Bu imza, sıralanabilecek basit veri tipleri olduğu sürece (karakterler, tam sayılar ya da büyüktür, küçüktür karşılaştırmasına girebilen başka tipler) yalnız girdi ve çıktı tipleri değiştirilerek başka veri türlerine kolayca uyarlanabilir.

5.2 Ön koşullar

Ön koşullar, girdiler hakkındaki savlardır, yani teoremlerdir. İlk adımda beklenen girdilerin var olup olmadığı kontrol edilir; biçimsel belirtimde bu kontrolün nasıl yapılacağı tanımlanır. Sıralamada yalnız girdinin tam sayı olması yeterlidir; sıfırdan büyük ya da küçük olmasıyla ilgili bir şey söylenmez. Bu durumda sıralama için ön koşul yazmaya gerek bile olmayabilir.

Karekök fonksiyonu yazılsaydı mutlaka bir ön koşul olurdu: negatif sayıların karekökü alınamayacağı için $x \geq 0$ yazılmalıdır. Veri tipi gerçel sayı değil de kompleks sayı olarak tanımlansaydı bu ön koşula ihtiyaç olmayabilirdi. Hoca burada "x negatif olursa imajiner sayılara geçerim"

gibi bir tanım yapmadığını vurgular; yalnız fonksiyonun ön koşullarının ve davranışının ne olması gerektiğini tanımlar. Negatif girdide istisna üreten bir sürüm de tanımlanabilirdi, ama hocaya göre ön koşulu garanti etmek daha doğru bir yaklaşımdır.

Veri tipleri zaten imzada (`Vector<int>`) yazıldığı için ön koşulda ayrıca ele alınmaz. Boş vektör verilirse ne olacağı da düşünülmelidir: boş kümenin sıralanmış hâli de boş kümedir. "x'in boyutu sıfırdan büyüktür" diye bir koşul konabilir, ama hocaya göre boş kümeyi de kapsamak daha doğrudur. Sonuç olarak sıralama için ya hiç ön koşul tanımlanmaz ya da `true` yazılır: her koşulda çalışır.

5.3 Son koşullar

Son koşullar, işlem çalışıp bittikten sonra ortamı ne hâle getirdiğimizi, neyi garanti ettiğimizi söyler. Slayttaki tanım (ekrandan, 00:20:00):

- Fonksiyon tarafından üretilen çıktı hakkında neyin mutlaka doğru olması gerektiğini söyler.
- Tipik olarak çıktının girdi ile nasıl ilişkili olduğunu anlatır.
- SQRT fonksiyonumuz için son koşul ne olabilir? $Y = SQRT(X)$ dersek döngüsel tanım yaratmış oluruz.
- Aralarındaki sayısal ilişkiyi tanımlamalıyız: $Y \cdot Y = X$. Bu, karekök çalıştıktan sonra doğru olması gereken ifadedir; buna son koşul denir.

Hoca, son koşulun her zaman "çıktı eşittir girdinin dönüşümü" biçiminde olmayabileceğini de söyler. Girdinin bir kısmıyla işlem yapıp çıktıya yansıtabilir, bir kısmını başka bir sisteme aktarabilir ya da bir veritabanına yazabilirsiniz. Saf fonksiyonlar girdiyi çıktıya dönüştürür, ama programlamada yan etkiler de vardır: global değişkenlerde yapılan değişiklikler, sonuçta gösterilmeyen işlemler, başka bir nesnede yaratılan değişiklik, veritabanına yazılan veriler. Bu yüzden $Y \cdot Y = X$ gibi bir ifade her zaman yazılamaz; son koşul, ortamda yaratılan toplam etkiyi tanımlar.

6. Sıralama için son koşullar

Sıralama imzasına dönüldüğünde, veri tipleriyle ilgili ifadeler zaten imzaya taşınmıştır. Geriye iki iş kalır: önce sıralı olma koşulu, sonra tekrar sayıları ve başka eleman bulunmaması.

6.1 Sıralılık

Y'deki her eleman için, kendisinden sonra gelen bir eleman varsa, o eleman baktığımız elemandan büyük ya da eşit olmalıdır. Son eleman için ayrıca bir şey söylenmez. Hoca bunu tümevarıma benzetir: n elemanlı bir girdide ilk n-1 elemanın her birinin takip edenle ilişkisi garanti edilirse son elemanın en büyük olduğu da kendiliğinden çıkar. Sıralama algoritmalarının analizinde de son adımda son elemana bakılmaz, çünkü en büyük eleman zaten sona atılmıştır.

Bunun için bir indis değişkenine ve $i + 1$ gibi bir ifadeye ihtiyaç vardır. Slayttaki ifade (ekrandan, 00:24:30):

$$\forall i : 1 \leq i \leq (n-1),\ y[i] \leq y[i+1]$$

Slayttaki açıklamalar:

- Eleman sayısı n gibi bir değişkenle belirlenir. n negatif olmamalıdır; i, 1..n arasında değer alır.
- Tanıma göre 1..n-1 arasındaki değerler için $i + 1$'deki elemanın büyük ya da eşit olması söylenir.
- $n = 0$ ise $n - 1$ negatiftir; $i := 1..n - 1$ aralığında hiç i olmayacaktır. "Saçma ama ifade doğru"; son koşulun doğruluğunu değiştirmez.
- Bu y'lerin ne olduğu ve girdiye nasıl bağlı olduğu henüz söylenmedi.

Hoca, üst sınırın neden $n - 1$ olduğunu da açıklar: n yazılırsa $i + 1$ yüzünden $n + 1$'e gidilir, yani vektörün dışına taşılır.

6.2 Sıralı mı? (ordered)

Hoca bir ara örnek verir: `ordered` adında, x tam sayı vektörünü alıp Boolean döndüren bir imza olsaydı ne olurdu? Tip kontrolü yapılmadığı için hangi x gelirse gelsin "bu sıralı mı?" diye sorulabilirdi; boş vektör de sıralıdır. Bu imzanın son koşulu, 1'den x'in boyutuna kadar bütün i'ler için $x[i] \leq x[i+1]$ ifadesidir. Bu her i için doğruysa çıktı true, değilse false olur. Ters sıralı (reverse ordered) için tek fark küçük eşit yerine büyük eşit kontrolüdür; eşitlik yine mutlaka bulunmalıdır.

Ancak sıralamanın kendisi daha fazla son koşul ister: X'teki her şey Y'de olacak, Y'deki her şey X'ten gelecek ve tekrar sayıları aynı olacak.

7. Permütasyon

Bu koşulların her biri tek tek yazılabilir, ama bu çok daha zordur. Hoca, matematikte bildiğimiz bir kavramın bu işi zaten gördüğünü söyler: permütasyon. Slayttaki tanım (ekrandan, 00:30:00):

- Permütasyonun tanımı: X'in elemanları Y'nin elemanlarının bir permütasyonu ise, X, Y ile aynı elemanlara sahiptir.
- Permütasyon iyi tanımlanmış bir matematiksel özellik olduğu için kendimiz yazmak zorunda kalmayız.
- `Bool Z = Permütasyon(Vector<int> X, Vector<int> Y)`
- Son koşulları biraz hilelidir; özel durumlara bölünür ve her durum ayrı ele alınır:
  - 1. durum: iki vektör de boş: $|X| = 0 \Rightarrow \text{Permütasyon}(X, Y)$.
  - 2. durum: vektörler boş değil. İlk elemanları aynı mı değil mi?
  - 2.1: aynı. X ve Y'nin kalan kısımları permütasyonu mu? $|X| > 0 \wedge X[1] = Y[1] \wedge \text{Permütasyon}(\text{kalan}(X), \text{kalan}(Y)) \Rightarrow \text{Permütasyon}(X, Y)$. Özyinelemelidir; kalan her zaman daha küçüktür ve sıfır uzunluk durumu ele alınmıştır.

Çıktı girdinin bir permütasyonuysa çıktıdaki tüm elemanlar girdiden gelir, her eleman çıktıda yer alır ve farklı sayıda yer alamaz. Hoca örnek verir: 1, 2, 3, 4 girdisinin permütasyonları 4, 2, 3, 1; 4, 3, 2, 1; 1, 2, 3, 4; 2, 1, 3, 4 gibi, dört sayının bütün dizilişleridir. Böylece "her şey X'ten gelecek, tekrarlar aynı olacak, X dışında eleman olmayacak" koşullarının üçü birden sağlanır.

İkinci durumda ilk elemanlar eşit değilse, hocanın örneğinde $X[1] = 5$ olsun: çıktının herhangi bir yerinde 5 bulunmalıdır, yoksa permütasyon değildir. Sonra Y, 5'in bulunduğu yere göre parçalanır ve önceki ve sonraki parçalarla X'in kalanı yine özyinelemeli olarak kontrol edilir. Hoca bunun ayrıntısına "çok fazla matematik ifadeye girmiş olacağız" diyerek uzun uzun girmez. Slayttaki birleşik son koşul (ekrandan, 00:33:00):

```
Permütasyon(X,Y) <=>
  |X| = 0  V
  ( |X| > 0 =>
      ( X[1] = Y[1] ∧ Permütasyon( kalan(X), kalan(Y) ) )  V
      ( X[1] ≠ Y[1] ∧
        ( ∃j: 1 < j ≤ |Y| • ( X[1] = Y[j] ) ∧
          Permütasyon( kalan(X), (Y[1..j-1] ++ Y[j+1..|Y|]) ) ) ) )
```

Hoca bunun "başkasının bizim için yazdığı" bir tanım olduğunu söyler. Böylece sıralamanın son koşulu doğrudan yazılabilir: her i için $y[i] \leq y[i+1]$ ve x ile y permütasyondur. Bu, başta Türkçe uzun bir cümleyle anlatılan bütün ayrıntıları birebir karşılar.

> **Ek bilgi (derste anlatılmadı):** Permütasyon koşulu çoğu zaman "çoklu küme (multiset) eşitliği" olarak da yazılır: her değer için X'teki ve Y'deki tekrar sayıları eşittir. Özyinelemeli tanımla aynı şeyi söyler; hangisinin seçileceği, kullanılan belirtim dilinin hazır işlemlerine bağlıdır.

Hoca, bunları çok hızlı aktardığını ama asıl mesajın şu olduğunu söyler: çıktının tekrar sayısının, hariçten eleman olmamasının, eleman sayısının girdiyle aynı olmasının resmî olarak tanımlanması gerekir; davranışın ve ortamın ne hâle getirildiğinin biçimsel tanımı son koşullarda yapılır.

8. Aynı belirtim OCL ile

Hoca, bu konuların aslında geçen hafta anlatılması gerektiğini ama ders yeterince uzun olduğu için orada bırakıldığını söyler ve aynı tanımın OCL ile nasıl yazılacağını gösterir. Slayttaki belirtim (ekrandan, 00:35:30):

```
context Vector::Sıralı() : Boolean
pre: true
post: self -> forAll (i:Integer | i > 0
        and i < self.length implies
        self.at[i] <= self.at[i+1]
```

Slayttaki açıklamalar: metodun argümanı yoktur; tüm nesneye dayalı dillerdeki gibi mesajın yollandığı nesne, belirtilen argüman olarak hizmet eder. Ön koşul true ise tanımlamaya gerek yoktur.

Hoca ifadeyi şöyle okur: `self` için `forAll`; i bir tam sayıdır, i sıfırdan büyüktür ve i `self.length`'ten küçüktür; bu durumda (implies) i konumundaki değer, $i + 1$ konumundaki değerden küçük veya eşittir. i, length'ten küçük olduğu için en son n. elemana bakılmış olur.

FOL ile arasındaki küçük farklar:

- | (pipe) sembolü OCL'de değişkeni önermeden ayırır; burada i'nin Integer bir değişken olduğu tanımlanıp önermeden ayrılır.
- i değişkeniyle ilgili kısıtlar (tam sayı olması gibi) önermenin bir parçası olarak yer alır.
- Döngü değişkeni gibi kullanıldığında `implies` anahtar kelimesi kullanılır; bu, "if-then" anlamındaki çıkarımdır.
- OCL'de vektör tanımı yoktur, sequence'lar vardır; hoca ikisini aynı şey gibi düşünmenin sorun olmadığını söyler.

Hoca bu bölümü şöyle bağlar: kesin işlevsellik tanımına ihtiyaç olduğunda biçimsel diller kullanılmalı ve biçimsel belirtim yazılmalıdır. Tam olarak ne üretmeye ve neyi garanti etmeye çalıştığınızı çok sıkı düşünmeniz gerekebilir. Bunu başta yapıp belirtimi kesin tanımlarsanız, harcadığınız emek sizi anlaşılmayan gereksinimler yüzünden tekrar tekrar çalışmaktan kurtarabilir. Sıralamada olduğu gibi "her çıktı bir öncekinden büyük olacak" demek sıralamayı garanti etmez; çıktıyla girdi arasındaki ilişki de tanımlanmalıdır.

9. OCL: UML'in resmî bir parçası

Ders yeni slayt destesiyle devam eder. Slayttaki maddeler (ekrandan, 00:40:00):

- Şimdiye kadar hep diyagramlara baktık. Ama diyagramlar her şeyi anlatamaz. Tanımlarda, tasarımda daha ayrıntı gerektiren noktalar vardır; bunun için OCL kullanılır.
- OCL bir belirtim (spesifikasyon) dilidir. Sistem özelliklerinin işlevsel ayrıntılarını tanımlamanızı sağlar.
  - Bildiren (declarative): yapılacak kodu ve adımları açıkça belirtmeden sonuçları tanımlar. İmperatifin tam tersidir.
  - Güçlü tipli (strongly typed): tüm değişkenler bellidir ve bir tip diğerinin yerine kullanılamaz. Weakly typed'ın tersidir.
  - Kısıtlar, koleksiyon sınıfları, diyagramlardaki sınıflar ve ilişkiler arası gezinti mekanizması.
- Pek çok araç OCL'i destekler: Rational Rose, ArgoUML, Eclipse, Poseidon/Octopus, Enterprise Architect.

Hoca, OCL öğrenildikten sonra geçen haftaki kütüphane örneğine ("Saffet'in kütüphanesi") dönüleceğini, örneğin 12 yaş altındaki çocukların en fazla 5 kitap alabilmesi gibi kısıtların OCL ile yazılacağını söyler.

İmperatif bir dilde adım adım işletilecek komutlar verilir. Bildirimsel dillerde ise yazılacak kod ya da algoritma adımları açıkça belirtilmez; sonuçların ne olacağı, ortamın ne hâle geleceği ve neyin garanti edileceği bildirilir.

Güçlü tiplemeyi açıklarken hoca zayıf tipli diller için JavaScript ve Python'u örnek verir: `i = 1` deyip kullandıktan sonra aşağıda `i = "ali"` diye metin atanabilir; aynı değişken bir yerde

tam sayı, bir yerde metin olur. C ve Java gibi dillerde ise değişkenin tipi değiştirilemez.

> **Ders sonrası not:** Hoca Python'u zayıf tipli, C'yi güçlü tipli örnek verdi. Python'da bir değişkene farklı tipte değer atanabilmesi dinamik tiplemedir; Python büyük olasılıkla güçlü tipli sayılır (`1 + "a"` hata verir). C statik tiplidir ama örtük tip dönüşümleri yüzünden çoğu kaynakta zayıf tipli kabul edilir. Hocanın kastettiği büyük olasılıkla statik ve dinamik tipleme ayrımıdır; OCL'deki anlamıyla önemli olan, her değişkenin tipinin belli olması ve değişmemesidir.

UML diyagramı çizilen araçların çoğu OCL yazmaya ve tanımları bildirmeye de izin verir. Hoca Rational Rose'un uzun süre çok meşhur olduğunu, hâlâ var olup olmadığını bilmediğini söyler. Bu tanımları bildirmek, örneğin bir davranışın başka türlü çalışmamasını garanti etmeyi sağlar.

10. OCL ile neler tanımlanır?

Hoca OCL'nin ne sağladığını şöyle sıralar: FOL'de gördüğümüz gibi değişmeyen bir sınıf yapısı tanımlanabilir; işlemlerin çalışması için gerekli ön koşullar ve çalışıp bittikten sonra ortamın ne hâle geldiği kesin olarak tanımlanabilir; durum diyagramlarında hangi koşullar altında bir durumdan başka bir duruma geçileceği de OCL ifadeleriyle tanımlanır. Durum modellemesi sonraki derste işlenecektir.

OCL'nin özellikleri bir kez daha toparlanır: bildirimseldir, prosedürel bir programlama dili değil tanım dilidir. Aktiviteleri değil değerleri tanımlar, oluşacakları belirler. Atama ifadesi yoktur; eşittir kullanılır ama bu bir bildirim, bir kısıt ya da özellik tanımıdır. Kendi veri tipleri ve UML diyagramlarında kullanılan veri tipleri vardır, bunların tipleri değişmez. OCL'de tanımlanan tek şey kısıtlardır; adı da zaten Object Constraint Language'dir.

OCL ile tanımlanabilecek kısıtlar:

- Sınıflarda değişmemesi gereken ifadeler (değişmezler): "illa böyle olacak" denenler.
- İşlemlerin ön ve son koşulları.
- Türetilen özelliklerin nasıl türetileceği. Örneğin önceki derste yanına `/` konan yaş özelliği, bugünden doğum tarihi çıkarılarak bulunur; faiz hesabı gibi başka türetilen özellikler daha karmaşık olabilir.
- Durum diyagramlarındaki (statechart) geçişler için guard'lar: hangi durumdan hangi duruma hangi koşulla geçileceği.
- Mesajlar için hedefler: bir mesajın hangi hedeflere gideceğinin kısıtlanması.
- Stereotipler için tip değişmezleri (type invariant).
- Bir sorgu dili gibi kullanılarak sınıf diyagramından türetilen nesnelerin belli kriterlere göre sorgulanması.

11. OCL sözdizimi ve kısıt tipleri

OCL'de yalnız kısıt yazıldığı için sözdizimi basittir:

```
context <tanımlayıcı> <kısıt tipi> : <Boolean ifade>
```

Yazılan bütün kısıtlarda önce bir context tanımlaması yapılmalıdır. Context, bir diyagramda, örneğin bir sınıf diyagramında, nerede durduğumuzu, odak ya da başlangıç noktamızın neresi olduğunu gösterir. Tanımlayıcı, context'in adıdır; genellikle bir sınıf adı verilir: `context Ziyaretçi`, "Ziyaretçi sınıfı kapsamında bir kısıt yazıyorum" demektir. Bir işlemle ilgili kısıt yazılacaksa metodun adı verilir: `Ziyaretçi::ödünçAldıkları`.

Üç kısıt tipi vardır:

- Değişmezler (invariant): her zaman doğru olan tanımlar.
- Ön koşullar (precondition): kısıtın doğru olması, işlemin çalışması için gerekli koşullar.
- Son koşullar (postcondition): işlem bittikten sonra neyin garanti edildiği, bildirildiği.

Sonrasında kısıtın tanımı olan mantıksal ifade gelir: büyüktür, küçüktür, permütasyondur gibi. Kısıtlar UML'deki sınıf modeliyle birebir ilişki içinde olmalıdır; bunun için önce bir sınıf modeli oluşturulmuş olmalıdır. Sınıf modelinde tanımlanamayanlar ve modelin üzerine kısıt olarak söylenmesi gerekenler OCL ile verilir.

12. Değişmezler (invariant)

Slayttaki tanım (ekrandan, 00:50:00):

- Bir özelliğin her zaman doğru olan ifadesidir.
- Sistemin anahtar bir gereksiniminin tanımı olarak düşünülebilir. Örneğin sistemdeki nesnelerin değerleri arasında gerekli bir ilişki.
- Anahtar kelime: `inv`.

```
context BüyükŞirket
inv: çalışanSayısı > 50
```

BüyükŞirket sınıfında çalışanSayısı özelliği 50'den büyük olmalıdır. Context BüyükŞirket olduğu için `BüyükŞirket.çalışanSayısı` yazmaya gerek yoktur; zaten oradayız, `self` burasıdır. Hoca bunu şöyle okur: "Eğer sen BüyükŞirket sınıfından türeyen, uygulamada kullanılan bir nesneysen, bu şartı her koşulda sağlamalısın." Çalışan sayısı 50'nin altına düşerse nesne BüyükŞirket sınıfına ait olamaz. Hoca, bu durumda nesneyi yok mu edeceğinizi yoksa çalışan sayısının 50'nin altına düşmesini mi engelleyeceğinizi sorar; aslında tanımlanan şey "50'den aşağı düşmemelisin" kısıtıdır.

Slayt değişmezleri veritabanındaki veri bütünlüğü kısıtlarına (integrity constraints) benzetir:

- İşçi şirkette çalışır. Şirket iflas etti, veritabanından silindi. İşçi nerede çalışır?
- Programlamadaki avare işaretçi (dangling pointer) durumu oluşur. Veritabanı yönetim sistemi bu durumu engeller.
- OCL değişmez kısıtları da sistem tanımlamalarında aynı rolü üstlenir.

Hoca, yönetici olarak işçileri de silebileceğinizi, ama normalde içinde işçi varken şirketin silinmesinin engellenmesinin beklendiğini söyler. C gibi dillerde avare işaretçi rahatlıkla yaratılabilir, ama veritabanı yönetim sistemleri bunu engeller.

13. Ön ve son koşullar

İkinci ve üçüncü kısıt tipleri olan ön ve son koşullar, işlemlerin anlamını verir; genellikle işlemlere yazılır. `pre` yazılıyorsa, işlemin anlamlı biçimde çalışması için ortamın ne hâlde olması, neyin doğru olması gerektiği söylenir. `post` ise işlem çalışıp bittiğinde ortamın ne hâle geleceğini, örneğin sıralamanın nasıl bir çıktı oluşmasını garanti ettiğini söyler. Böylece dönüş değeriyle girdiler arasındaki ilişki tanımlanır; işlem sınıfların özelliklerinde ya da ortamda başka değişiklikler yaratıyorsa bunlar da tanımlanmalıdır.

Bir context için pre ve post birlikte yazılabilir ya da ayrı kısıtlarda tanımlanabilir. Hoca bunun çok önemli olmadığını, ama birlikte yazmanın genellikle daha okunaklı olduğunu söyler.

FOL'de yapılan karekök örneğinin OCL hâli şöyle anlatılır: context, tam sayı sınıfında tanımlı karekök işlemidir; bir tam sayı çıktı üretecektir. Ön koşul `self >= 0`'dır; `self`, context'in kendisidir. Bu sağlanmazsa işlem çalışmaz, yani "negatif sayılarda çalışmam" denmiş olur. Son koşul `self = result * result`'tır; `result`, dönüş değeridir. Böylece FOL'deki $Y \cdot Y = X$ tanımı OCL'de yazılmış olur.

Hoca burada tamamen bildirim yapıldığını vurgular: karekökün nasıl alınacağı, hangi atamanın yapılacağı, neyin çarpılıp bölüneceği adım adım, imperatif biçimde söylenmez; yalnız bildirilir.

> **Ders sonrası not:** Örnekte dönüş tipi tam sayı olarak anlatıldı. `self = result * result` son koşulu yalnız tam kare sayılar için sağlanabilir; örneğin 2'nin tam sayı karekökü yoktur. Genel bir karekök için dönüş tipinin Real olması ve son koşulun bir yaklaşıklık payıyla yazılması büyük olasılıkla daha uygundur.

14. Özellik değerlerinde değişiklik ve @pre

İşlemler saf fonksiyon olmayabilir; özelliklerin değerlerini değiştirebilir. Hocanın örneği bir banka hesabıdır: hesabın bakiye (balance) özelliği vardır ve para yatırma (deposit) ya da çekme (withdraw) işlemleri yapılabilir. Slayttaki örnek (ekrandan, 00:59:05):

```
context Account::deposit(Real : amount)
pre:  amount > 0
post: balance = balance + amount
```

Context, Account sınıfının deposit davranışıdır. Girdi, amount adında bir gerçel sayıdır. Hoca, negatif bir değer verilirse “para mı çekmeye çalışıyoruz?” sorusunun akla gelebileceğini, ama para çekmede başka kontrollerin de gerekeceğini, bu yüzden deposit’in yalnız sıfırdan büyük değerler için çalışması gerektiğini söyler. Ön koşul budur; negatif bir sayı gelirse deposit hiç çalışmaz.

Son koşul mantık olarak “bakiye, bakiye artı tutardır” gibi görünür, ama bu bir atamadır. OCL’de atama yoktur, = eşitliktir. Hoca bunu şöyle gösterir: eşitliğin iki yanındaki `balance` sadeleştirilirse geriye “amount sıfıra eşittir” gibi bir ifade kalır; bu doğru bir ifade değildir. Slayttaki uyarı: “OCL bildirimsel bir dildir. Buradaki = işareti atama değil, eşitliktir! İfade yanlış olur. Değer değiştiriyorsak @pre kullanmalıyız; işlem çalışmadan önceki değer.” Doğrusu:

```
post: balance = balance@pre + amount
```

Artık bu bir eşitliktir: önceki değer artı gelen tutar, yeni değere eşittir. Böylece ortamda bir değişiklik yaratılmış, nesnenin bir özelliği değiştirilmiş olur.

14.1 Swap örneği

Hoca bir örnek sorar (ekrandan, 01:00:00): a ve b adlı Real tipinde iki özelliği olan bir sınıf olsun. Sınıfta swap adlı bir işlem a ile b’nin değerlerini takas etsin. Son koşul `post: ___ = ___ and ___ = ___` biçiminde nasıl yazılır?

Akla ilk gelen, `tmp = a; a = b; b = tmp` gibi değişken atamalarıdır, ama OCL’de atama yoktur. Hoca cevabı şöyle verir:

```
post: a = b@pre and b = a@pre
```

a değişkeni b’nin ilk hâline eşittir ve bununla birlikte b değişkeni a’nın ilk hâline eşittir. Programlamada mutlaka geçici bir değişken kullanılır, ama burada buna gerek yoktur. Bu noktada derse ara verilir.

15. OCL veri tipleri ve anahtar kelimeleri

15.1 Basit veri tipleri

Aradan sonra hoca OCL’nin sunduğu veri tiplerine geçer. Slayttaki tablo (ekrandan, 01:10:00):

Tip	Değerler	İşlemler
Boolean	true, false	and, or, xor, not, implies, if-then-else
Integer	1, -5, 2, 34, 26524, …	*, +, -, /, `abs()`, …
Real	1.2, 3.14, -45.678, …	*, +, -, /, `floor()`, …
String	‘merhaba dünya’	`toUpper()`, `concat()`

Hoca tablonun bu kadarla sınırlı olmadığını, hangi tiple hangi işlemlerin yapılabildiğinin araştırılması gerektiğini söyler. Gerçel sayılarda yukarı ve aşağı yuvarlama gibi, metinlerde büyük ve küçük harfe çevirme, birleştirme ve alt metin alma gibi hazır işlemler vardır.

15.2 Anahtar kelimeler

OCL’de küçük bir anahtar kelime kümesi vardır. Slayttaki tablo (ekrandan, 01:20:00):

Açıklama	Anahtar kelime	Tanım
Bunları gördük	`inv`, `pre`, `post`	Kısıtları tanımlar
Koşullu ifadeler için	`if-then-else-endif`	Koşul ifadesi

Açıklama	Anahtar kelime	Tanım
Mantıksal operatörler	`not`, `or`, `and`, `xor`, `implies`	Mantıksal operatörler
UML'in birimleri bölütleme yeteneğini gösteren paketleme	`package`, `endpackage`	Paketler
Bunu da gördük	`context`	Kısıtları tanımlar
Tanım yapmamızı sağlayan çeşitli kelimeler	`def`	Global tanım
Bunlarla kısa hâllerini kullanabiliriz	`let`, `in`	Lokal tanım
Hesaplanan özellik değerini tanımlayabiliriz	`derive`	Özellik türetme
İlk değer atamasını init ile yapabiliriz	`init`	İlk değer tanımı
Bunu da gördük	`result`	İşlemin sonuç değeri
Bunu da gördük	`self`	Kısıtlanan nesnenin kendisi

Hocanın açıklamaları: koşullu ifadelerde else-if zincirleri de kurulabilir. Mantıksal operatörler yazıyla yazılır; `implies`, "p ise q" çıkarımıdır. `package` ve `endpackage` bir paketin başlangıcını ve sonunu tanımlar; bu derste pek kullanılmayacaktır. `context` her zaman ilk yazılan kelimedir. `def` global tanım içindir. `let` ve `in`, uzun bir ifadeye kısa bir takma ad verip onu birkaç yerde kullanmayı sağlar. `derive` hesaplanan özellikleri, `init` ilk değer atamasını tanımlar. `result` dönüş değeri, `self` ise context içindeki, o anda kısıtlanan nesnedir. Hoca dilin çok karmaşık olmadığını, ama bu küçük anahtar kelime havuzuyla her türlü kısıtın diyagramlarla eşlenik olarak yazılabildiğini söyler.

16. Let ifadesi

Slayttaki açıklama ve örnek (ekrandan, 01:24:20):

- Let, yerel kısaltma ya da tanım yapabilmemizi sağlar.
- Kısıtların birinde birden fazla kez kullanılacak karmaşık bir hesaplama olsun. Her seferinde tekrar yazılabilir ama hata olasılığı yüksektir.
- Let ifadesi ile ifadenin değerini taşıyan yeni bir tanımlayıcı türetilebilir ve kısıtlarda kullanılabilir.
- Örnek: income, işlerimizdeki tüm maaşlarımızın toplamı olsun.

```
let income : Integer = self.job.salary->sum() in
    if isUnemployed then income < 100
                    else income >= 100
endif
```

Hoca bunu şöyle okur: context olarak örneğin bir çalışan nesnesindeyiz; `self.job.salary->sum()`, benim üzerimden erişilebilen job nesnelerindeki salary özelliklerinin toplamıdır. `let income ... in` dedikten sonra kısıt yazılır: işsizse gelir 100'den küçük olmalıdır, değilse 100'den büyük ya da eşit olmalıdır. Let kullanılmasaydı `self.job.salary->sum()` ifadesi koşulun her iki kolunda tekrar tekrar yazılmak zorunda kalınırdı. Hoca let'i makro yazar gibi ifadeleri sadeleştirmek için kullanılan yerel bir kısaltma olarak tanımlar.

17. Gezinti, çoğullama ve koleksiyonlar

17.1 Gezinti (navigation)

OCL'de UML diyagramları üzerinde de kullanılabilen bir gezinti mekanizması vardır. Yazılan her OCL kısıtı bir sınıf diyagramıyla ilişki içindedir ve mutlaka bir sınıf ya da işlemle başlar. Ama yalnız context sınıfının değil, onunla ilişkili başka sınıfların özelliklerine de ulaşmak gerekir. Bir değişmezde context dışındaki bir sınıfın özelliğine referans vermek için nitelikli adlar kullanılır: diyagram üzerinde yürünerek, her noktada yeni bir context'e geçilerek, ilişkideki

başka bir sınıfa, o sınıfın bir ilişkisine ya da ilişkiye ait bir özelliğe kadar gidilir. Bunun ilk şartı, diyagramda oraya erişilebilir olmaktır.

Slayttaki örnek (ekrandan, 01:30:00): Customer sınıfı context'tir; bir siparişin hangi tarihte verildiğiyle ilgili bir kısıt yazılacaktır. Slayttaki sınıf diyagramı metne şöyle dökülebilir:

```
Customer (name, address)            1 ---- *  Order (date, status; calcTax(), calcTotal(), cal
Order                               1 ---- 1..*  Payment (amount)
Payment <|-- Credit (number, type, expDate)
Payment <|-- Cash (cashTendered)
Payment <|-- Check (name, bankID)
Credit, Check ..|> <<interface>> Authorization (authorized)
Order  <>---- OrderDetail (quantity, taxStatus; calcSubTotal(), calcWeight())
OrderDetail  ---- Item (shippingWeight, description; getPriceForQuantity(), getWeight())
```

Hoca, belki bir yıldan önce ya da sonra sipariş verilememesi gibi bir kısıt düşünür. Customer'dayız; burası `self`'tir. `self.` dendiğinde Customer'dan erişilebilenlere gidilebilir; bu diyagramda yalnız Order'a gidilebilir. `self.order` dendiğinde artık Order context'indeyiz; Order'ın bir date özelliği vardır. `self.order.date` ile, Customer context'inde olunmasına rağmen başka bir yerdeki özellik kısıtın içinde kullanılabilir.

17.2 Çoğullama

Sınıf diyagramlarında sayılar ve * ile bir sınıftan kaç örneğin diğer sınıftan kaç örnekle ilişkide olabileceği tanımlanıyordu. Slayttaki örnekler (ekrandan, 01:31:20): 1-1 gelin/damat, 1-n anne/çocuk, m-n öğrenci/ders. UML'de çoğullama kullanıldığında gezintinin sonucu tek bir nesne değil, bir koleksiyondur: küme, yığın ya da sıralı dizi olabilir. OCL bu koleksiyonlar üzerinde işlem yapmaya izin verir. `->` sembolü, koleksiyonun adını işlemin adından ayırır; hoca bunu C'de struct'lara erişmeye benzetir.

Slayttaki soru: önceki örnekte ödeme birden fazla bankanın çeki ile yapılsın. Sayı nasıl bulunur? (Koleksiyonda `size()` işlemi vardır.) Customer nesnesindeyiz:

```
self.Order.Check.bankID->size()
```

Hoca neden Payment yerine doğrudan Check'e gidildiğini açıklar: Order, Payment ile ilişkilidir ama önemli olan, genelleştirme nedeniyle ödemenin Credit, Cash ya da Check olmasıdır. Bu yüzden `Order.Check` diye devam edilir. Çoğullamada * ya da birden büyük bir sayı varsa mutlaka koleksiyon döner; o zaman kısıt koleksiyon nesneleri üzerinde yazılmalıdır.

> **Ders sonrası not:** Hoca ifadeyi "kaç farklı banka çeki ile ödeme yaptım" diye okudu. Gezintinin sonucu tekrarlı bir koleksiyon (bag) olduğundan `size()` büyük olasılıkla çek sayısını verir; aynı bankadan iki çek varsa iki kez sayılır. Farklı banka sayısı için büyük olasılıkla `self.Order.Check.bankID->asSet()->size()` yazılmalıdır.

17.3 Koleksiyon tipleri ve işlemleri

Hoca OCL'nin üç ana elemanını şöyle toparlar: birincisi kısıtın kendisi, ikincisi gezinti (context değiştirme), üçüncüsü koleksiyon sınıflarıdır. 1-n ya da n-m ilişkileri destekleyen dört koleksiyon çeşidi vardır:

- Collection: genel koleksiyon.
- Set: küme; bir eleman yalnız bir kez bulunur, tekrar olmaz.
- Bag: tekrar içerebilen koleksiyon.
- Sequence: sıralı dizi; girdilerin sıralı olmasını garanti eden özel bir koleksiyon tipi.

Collection üzerindeki işlemler: `size` (kaç eleman var), `includes` (belli bir elemanı içeriyor mu), `count` (sayısı), `includesAll` (hepsini içeriyor mu), boş mu dolu mu kontrolleri, `sum` (toplam değer), `exists`, `forAll` ve `iterate`. Son üçü bütün nesneleri tek tek gezerek işlem yapmayı, bir for döngüsü kurmayı sağlar. Set, Bag ve Sequence bu işlemleri Collection'dan miras alır ve kendilerine özel işlemleri de vardır. Hoca bunların ayrıntısı için OCL başvuru kılavuzuna bakılmasını önerir; bu bir ileri seviye ders olduğu için farkında olmak yeterlidir.

> **Ek bilgi (derste anlatılmadı):** OCL'de koleksiyon işlemleri `->` ile, nesne özellikleri ve işlemleri `.` ile çağrılır. Örneğin `self.order` bir koleksiyon döndürüyorsa

`self.order->size()` sipariş sayısını, `self.order->forAll(o | o.status = ...)` ise her sipariş için bir koşulu ifade eder. Derste kullanılan `including`, `reject` ve `includes` da bu tür koleksiyon işlemleridir.

18. Diğer OCL özellikleri

Kısıtlar, gezinti ve koleksiyonlar dışında daha az kullanılan özellikler de vardır:

- Tuple: bir kayıt (record) yazar gibi, bir değişken grubu oluşturmayı sağlar.
- Enumeration: Java'daki ya da diğer dillerdeki gibi sayılamalar.
- Mesaj ifadeleri: bir sinyalin gönderildiğine dair belirtim yaratmak ya da bir işlemi tetiklemek.
- UML meta modeline erişim: UML meta modeli değiştirilip kendi UML'inizi yaratabileceğiniz önceki derslerde söylenmişti; bunun için de OCL kısıtları yazılabilir. Hoca bunu en ileri seviye olarak niteler.
- Otomatik düzleştirme (automatic flattening): üzerinde pek çok bileşen olan iki ilişki boyunca gezinti yapılıyorsa (kümelerin kümesi, kümelerin yığını gibi) iki seviyeyi ayrı ayrı belirtmeye gerek yoktur; doğrudan ikinci seviyeden söz etmek yeterlidir.

Hoca OCL'yi şöyle özetler: FOL'e göre görece kolay bir dildir; aynı şey yazılır, ama programlama mantığıyla yazıldığı için uygulaması daha kolaydır. CASE araçları (Eclipse ve eklentileri, ArgoUML, Enterprise Architect) destek verir. Sistemin özelliklerini kesin olarak tanımlamayı sağlar; yapısal ve davranışsal diyagramların vazgeçilmez tamamlayıcısıdır, UML'in ayrılmaz bir parçasıdır. Kesin tanımlı olması gereken durumlarda mutlaka yazılması gerekir.

19. Kütüphane örneğine dönüş: hangi gereksinim OCL ister?

Son bölümde hoca "Saffet'in kütüphanesine" döner ve UML diyagramıyla gösterilemeyen gereksinimlerin OCL ile nasıl ifade edileceğine, hepsine değil ama birkaçına bakar. Gereksinimler hatırlatılır:

- Her ziyaretçinin tekil bir kütüphane kartı vardır.
- Kütüphane ad, adres, telefon ve kart numarasını bilmelidir.
- Herhangi bir anda ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli ya da hesaplayabilmelidir. Hoca burada "buram buram OCL koktuğunu" söyler: ödünç alınanları almak, teslim zamanlarını bilmek ve bunlara göre ceza hesabı yapmak gerekir; hesabın nasıl yapılacağı OCL ile tanımlanabilir.
- 12 yaş altındakiler en fazla 5 parça alabilir. Hoca "Hoppa! Direkt olarak invariant" der; son koşulla da yapılabileceğini, tek doğru yolun olmadığını, ama her zaman en basitinin tercih edilmesi gerektiğini ekler.
- Kitap ve DVD ödünç alınabilir; kitaplar 3 hafta, çok satanlar 2 hafta, DVD'ler 2 hafta verilir; gecikme cezası 10 liradır. Böyle sayılar varsa bunların değişmez olarak tanımlanması beklenir.
- Ödünç alınamayan parçalar vardır; bu sınıf modelinde ödünç verilebilir ve verilemez parçaları ayırarak zaten çözülmüştü.
- Talep etme ve süre uzatma ile ilgili de kısıtlar gerekebilir.

Sınıf modeli de hatırlatılır: ziyaretçiler başlıkları talep eder. Don Quixote'un beş ayrı kopyası olabilir; illa dördüncü kopya istenmez, yalnız Don Quixote istenir ve bir tanesi verilir. Ödünç ilişki sınıfının uzatıldı mı, teslim tarihi ve türetilen ceza özellikleri vardır; ceza hesaplanan bir özellik olduğu için OCL yazılması beklenir. Başlıklar parçalardan oluşur; parçalar ödünç verilebilir ya da verilemez. Verilemeyenler başvuru kitabı ve dergi, verilebilenler DVD ve kitaptır. Kitabın çok satan olup olmama durumu vardır. Para, tarih ve işletim sistemi gibi hizmet sınıfları `<<utility>>` stereotipiyle gösterilmişti.

Slayt hangi gereksinimin yeterince aydınlatıldığını tartışır (ekrandan, 01:40:00):

- 2. Kütüphane her ziyaretçi için en az ad, adres, telefon, kütüphane kartı numarasını bilmelidir. Gereksinimler bilgilerin varlığı gibiyse, bunlar diyagramda (özelliklerde) yeterince anlatılır.
- 3. Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir. Burada bil-

giyi üretmek için sorgular yapılmalıdır. Bir işlem varsa diyagramda bunun imzası verilebilir; hesaplama OCL ile tanımlanır.

- 9. Kütüphanede ödünç alınamayan başvuru kitapları ve dergiler de vardır. Burada da bilgi var ve genelleme ile tüm bilgi sağlandı.
- 5. Ziyaretçi kitap veya DVD ödünç alabilir. Burada da bilgi var, ama nasıl ödünç alınamayacağı anlatılamıyor. Ödünç alma işleminin ayrıntıları verilmedi; örneğin çocuk ise limit var.
- Sınırlama: ya diyagramdaki tanım desteklenmeli ya da yeni bir tanım getirilmelidir. Bunlar OCL ifadeleri olacaktır.

20. Gereksinim 6: ödünç periyodu

Hoca basitten başlar: “Sayı görüyorsanız kesin değişmez tanımı yapmanız beklenir.” Altıncı gereksinim kitapların 3 hafta, çok satanların 2 hafta ödünç alınabileceğini söyler.

İlk adım, OCL ifadesinin hangi context’e, yani sınıf diyagramındaki hangi sınıfa yerleştirileceğine karar vermektir. Çok satan da kitabın bir özelliği olduğu için context Kitap olmalıdır. İkinci adım kısıt tipine karar vermektir: kısıt sistemin geneliyle ya da context sınıfıyla ilgiliyse, yani her zaman doğru olması gereken bir koşulsa değişmez yazılır; bir işlemle ilgiliyse ön ve son koşul olarak verilir. Burada iki seçenek vardır: Kitap sınıfına bir değişmez yazmak ya da yalnız ödünç alma işleminde kullanıldığı için ödünç alma işlemine bir son koşul eklemek. Hocaya göre kesin sayılar varsa bu limitler hep olacağı için değişmez tanımı daha mantıklıdır.

Slayttaki kısıt (ekrandan, 01:43:40):

```
context Kitap inv:
    if cokSatan then
        oduncPeriyodu = 2 --hafta
    else
        oduncPeriyodu = 3
    endif
```

Slayttaki açıklama: koşullu ifade programlama dillerindeki deyim gibi değildir; durum değişikliği yerine değer üretir. İki değer olasılığı vardır ve ikili ifadenin (cokSatan) değerine göre oduncPeriyodu ya 2 ya 3’tür.

Hoca, ödünç periyodunun önceki derste Kitap sınıfında görülen bir özellik olduğunu hatırlatır. OCL yorumları `--` ile yazılır. Böylece Kitap sınıfından türeyen herhangi bir nesnenin ödünç periyodu ya 2 ya 3’tür; çok satan true ise 2’dir. 2 ve 3 açıkça kullanılmıştır; bunların hafta olduğu bilinir, ama gerekirse Tarih sınıfı türünden daha düzgün bir ifade yazılabilir. Her kısıt o sınıf kapsamında yorumlanır: noktasız görülen, gezinti yapılmayan her şey o sınıfın kapsamındadır.

DVD’ler de iki hafta için ödünç verilir; burada şart yoktur ve context doğrudan DVD’dir:

```
context DVD inv: oduncPeriyodu = 2
```

Ödünç periyodu başka bir değer alamaz; sabit bir değer tanımlamak için de değişmez kullanılabilir.

21. Gereksinim 3: ödünç alınanlar

Üçüncü gereksinimde bir sorgu vardır: hiçbir şey değiştirilmez, özelliklerin değerleri sorgulanır. Bu işlemler de ön ve son koşullarla tanımlanabilir. Önceki derste Ziyaretçi sınıfına `ödünçAldıkları` diye bir metot yazılmıştı; dönüş değeri olarak yalnız bu ziyaretçinin ödünç aldıkları listelenmelidir.

Context, Ziyaretçi sınıfının ödünçAldıkları metodudur. Dönüş değeri ödünç verilebilir parçalardan oluşan bir kümedir. Argümanı yoktur; hoca görünmeyen bir argüman (örneğin global bir değişken) da olabileceğini söyler. Kümenin seçilmesinin nedeni şudur: bir başlığın pek çok kopyası ödünç alınabilir, örneğin üç ayrı Don Quixote kopyası. Ama alınan belirli bir kopya (rafta duran 4 numaralı etiketli Don Quixote) aynı anda iki kez alınamaz. Küme aynı elemanın iki kez bulunmamasını garanti eder.

Hoca bir durum daha düşünür: bir kitap ödünç alındı, teslim zamanı geçti, borç oluştu, sonra geri getirildi. Sistemin bunu ödünçler arasında görmesi, kaydın orada olması gerekir; küme koleksiyonu bu problemi de çözmeye yardımcı olur.

Ön koşul yoktur, çünkü gereksinim "herhangi bir anda" sorgulanabilmesini ister. Hocaya göre durum değişikliği yapmayan, yalnız sorgu yapan işlemlerde genellikle ön koşul olmaz; pre'ye true yazmak ya da hiç kullanmamak aynı şeydir.

Son koşulda dönen kümenin içeriği belirlenir. Slayttaki ifade (ekrandan, 01:49:30):

```
context Ziyaretci::oduncAldiklari() :
    Set(OduncVerilebilirParca)
post:
    result = Odunc.oduncVerilebilirParca
```

Slayta göre ödünç ilişkisi üzerinden bu ziyaretçinin aldığı ödünç alınabilir parçalara erişilir: hem ödünç alıp henüz getirmediklerimiz hem de ödünç alıp zamanını geçirip geç getirdiklerimiz. Hoca burada `self.Odunc` yazmanın daha doğru olacağını belirtir: benim üzerimden erişilen ödünç ilişki sınıfındaki ödünç verilebilir parça tipindeki nesnelerin tamamı sonucu oluşturur.

22. Gereksinim 4: çocuklar için sınır

Slayttaki başlık: "Gereksinim 4: 12 yaş altındaki çocuklar için özel bir kısıtlama vardır. Bir kerede en fazla 5 parça ödünç alabilirler." (ekrandan, 01:52:20)

Değişmez mi yoksa ön koşul mu yazılmalıdır? Slayttaki cevap: tüm ziyaretçiler için kısıt yok, tüm çocuklar için kısıt var; çocuktaki kontrol her ödünç alma işleminde yapılmalıdır. Hoca bunu şöyle açıklar: bir çocuk önce iki kitap alır, iki gün sonra gelip dört kitap daha alır ve altı kitaba ulaşarak 5 parça kısıtını ezebilir. Bu yüzden her ödünç alma işleminden önce kontrol yapılmalı, kontrol doğru değilse ödünç alma metodu hiç çalışmamalıdır.

```
context Ziyaretci::oduncAl(i: oduncVerilebilirParca)
    pre: yas <= 12 implies oduncAldiklari() -> size() < 5
```

Context, Ziyaretçi'nin ödünç al davranışıdır; parametre i, ödünç verilebilir bir parçadır (bir kitap ya da DVD). Ziyaretçi context'inde olunduğu için "çocuk ise" doğrudan `yas <= 12` diye yazılır. 12'den büyükse sağ tarafın önemi yoktur; 12'den küçükse sağ taraf doğru olmak zorundadır: "if P then Q", yani FOL'deki çıkarım. Sağ tarafta önceki bölümde tanımlanan `oduncAldiklari()` kümesinin boyutu kullanılır; 1-n ilişki olduğu için koleksiyon işlemi `->` ile çağrılır.

Slayttaki diğer notlar: ön koşul doğru olmazsa işlem çalışmayacak; koşul, alınanlar kümesinin boyutunun 5'ten küçük olmasıdır, "DİKKAT!! <= değil"; bu kısıt çocukların kaç tane ödünç aldıklarını açıkça göstermez ama asla 5'ten fazla olmamasını garanti altına alır; post kısmı yoktur, şu anda tam bir tanım değildir.

> **Hocanın vurgusu:** Küçük eşit olursa altıncı parçayı alma tehlikesi vardır. Daha önce 5 parça almış bir çocuk altıncı için geldiğinde `< 5` şartını sağlamaz.

Hoca bu noktada "toplamda değil, bir kerede en fazla beş" ayrımını da tartışır.

> **Ders sonrası not:** Hoca anlatımın bir yerinde "2 aldım, 4 aldım, 6 tane kitap aldım ama hiçbir zaman bir kerede 5 tane almadım" dedi. `oduncAl` her çağrıda tek bir parça (i) aldığı ve ön koşul her çağrıdan önce kontrol edildiği için, önceki parçalar iade edilmeden altıncı parça alınamaz; slayttaki "asla 5'ten fazla olmamasını garanti altına alır" cümlesi bu durumu anlatır. Python ile yapılan küçük bir benzetimde `< 5` ile elde tutulan parça sayısı en fazla 5'e, `<= 5` ile 6'ya ulaştı. Ayrıca slayt "12 yaş altındaki" derken OCL'de `yas <= 12` kullanıldı; bu, 12 yaşındakileri de kapsar. Gereksinime harfiyen uymak için büyük olasılıkla `yas < 12` yazılmalıdır.

23. Gereksinim 5: ödünç alma işlemi

Şimdiye kadar hep sorgu yapıldı: değerlere bakıldı, değerlere göre izin verildi ya da kısıt konuldu. Beşinci gereksinim (ziyaretçinin kitap ya da DVD ödünç alabilmesi) ise elemanların

durumunu değiştirir; ödünç alma işlemi son koşullarla tanımlanmalıdır. Bir context içinde (Ziyaretçi'nin ödünçAl'ı) birden fazla pre ve post kısıtı varsa bunlar ayrı ayrı tanımlanabilir ya da `and` ile birleştirilebilir.

23.1 Ön koşullar

Ödünç almanın ön koşulları:

- 12 yaşından küçükse aynı anda en fazla 5 parça (önceki bölüm).
- Ödünç alınabilir parçanın var olması gerekir. Bunun sorgulanabilmesi için "mevcut mu" diye bir davranış olmalıdır; yoksa parça alınamaz.
- İstek listesi vardı: ziyaretçi, istek listesindeki ilk kişi olmalıdır. Beşinci kişiyse "dur, senden önce dört kişi var" denmeli ve işlem çalışmamalıdır. Bunun için de "başkası istemedi mi", "sıradaki ilk sen misin" gibi bir davranış gerekir.

Hoca, bunu yaparken bir algoritmayı adım adım parçalar gibi işlemlerin eklendiğini ve ödünç almanın gerçekleşebilmesi için başka işlemlere de ihtiyaç olduğunun görüldüğünü söyler.

23.2 Son koşul 1: yeni ödünç nesnesi

Hocaya göre ön koşul yazmak genellikle daha kolaydır ("bunlar yoksa oynamam" demek gibi), son koşullar daha karmaşıktır. Ödünç alma bittiğinde sistemde neler olur? Slayttaki maddeler (ekrandan, 01:59:00):

1. oduncAl ilişkisinde yeni bir bağlantı (yeni bir nesne örneği) yaratılacak ve bize döndürülecek.
2. İstek listesini güncellememiz gerekir (eğer biz talep etmiştiysek ve ödünç aldıysak); almamız için listenin en başında olmamız gerekliydi.

```
post: exists(
    o: Odunc |
    o.oduncVerilebilirParca = i and
    o.teslimTarihi = os.getDate() + o.oduncVerilebilirParca.oduncPeriyodu and
    Odunc = Odunc@pre -> including(o) )
```

Hoca ifadeyi şöyle okur: bir ödünç nesnesi tanımlanır ve pipe ile ayrılır. Her şey bir `exists` içine alınır; `exists`, FOL'deki ters E'dir: "öyle bir ödünç nesnesi vardır ki bu ifade doğrudur." Bütün ödünç nesneleri için değil, en az biri için doğru olduğu garanti edildiği için `exists` kullanılmak zorundadır. Bu nesnenin ödünç verilebilir parçası, imzada gelen parametre i ile aynıdır. Teslim tarihi, işletim sisteminden alınan tarihe parçanın ödünç periyodunun (başka bir kısıttan gelen iki ya da üç hafta) eklenmesiyle bulunur. Son olarak ödünç koleksiyonu, önceki hâline bu yeni nesnenin eklenmiş hâli olur. Kısaca, argümanı i olan bir ödünç nesnesi yaratılır (Don Quixote ise Don Quixote için), teslim tarihi ayarlanır ve işlem bittiğinde bu nesne ilişki kümesine eklenir.

23.3 Son koşul 2: talep listesini güncellemek

Talep listesinin de güncellenmesi gerekir. İfade uzun olacağı için `let` ile kısaltılır. Bu, aynı context'e (Ziyaretçi'nin ödünçAl'ı) yazılır; `and` ile önceki son koşula eklenebilir ya da uzun olduğu için ayrı bir son koşul olarak yazılabilir. Slayttaki ifade (ekrandan, 02:01:50):

```
context Ziyaretci::oduncAl(i: oduncVerilebilirParca)
post: let b : Baslik = i.baslik in
    if b.talepler.ziyaretci -> includes(self)
    then talepler=talepler@pre->reject(baslik=b and ziyaretci=self)
    else
        true
    endif
```

Slayttaki açıklama: Let ifadesi ile mevcut parçanın başlığı kısaltılıp kullanılır. Eğer bu başlığa ait talebi yapan ziyaretçi biz isek, talep listesinden kendimizi çıkarırız. Implies ile de yapılabilirdi.

Hoca adımları şöyle anlatır: gelen parça örneğin bir Don Quixote kopyasıdır; `let b : Baslik = i.baslik`, bu kopyanın başlığıdır. Başlıklara talep yapılabiliyordu; sınıf diyagramına bakıldığında `b.talepler.ziyaretci` ile ziyaretçiye geçilir ve `includes(self)` ile bu ziyaretçinin talep listesinde olup olmadığına bakılır. Varsa talepler, ilk hâlinden bu başlık ve bu ziyaretçiye ait talebin `reject` ile atılmış hâline eşit olur; `reject` de hazır bir koleksiyon işlemidir. Yoksa hiçbir şey yapılmaz, ifade doğrudur. else kolu yalnız true olduğu için aynı şey `implies` ile de yazılabilirdi: "b'nin taleplerindeki ziyaretçiler self'i içeriyorsa, talepler eşittir …".

> **Ek bilgi (derste anlatılmadı):** `if A then B else true endif` ifadesi mantıksal olarak `A implies B` ile denktir, çünkü A yanlış olduğunda iki ifade de doğrudur, A doğru olduğunda ikisi de B'nin değerini alır. Bu yüzden else kolu true olan koşullu son koşullar genellikle `implies` ile daha kısa yazılır.

Hoca, basit bir iş olmasına rağmen burada bile küçük ayrıntıların düşünülmesi gerektiğini söyler.

24. Türetilen özellik: yaş

Veri türetmek için de OCL kullanılır. Bu sefer context bir özelliktir. Türetilen özelliklerde inv, pre ya da post değil, `derive` kullanılır. Slayttaki ifade (ekrandan, 02:03:40):

```
context Ziyaretci::yas: Date
  derive:
    OperatingSystem.getDate() - doğumTarihi
```

Bugünün tarihinden, ziyaretçinin içindeki bir tarih olan doğum tarihi çıkarılır ve yaş özelliği bununla türetilir. Böylece yaşın, her sorgulandığında bu tanımla güncellenmesi, türetilmesi gereken bir özellik olduğu OCL ile bildirilmiş olur.

> **Ders sonrası not:** Slaytta yaş `Date` tipiyle yazıldı ve hoca da "date tipinde olması lazım" dedi; önceki dersin son modelinde ise `/yaş: Integer` idi ve Gereksinim 4'teki `yas <= 12` karşılaştırması da tam sayı ister. İki tarihin farkından yaş elde etmek için büyük olasılıkla Integer tipi ve yıl hesabı gerekir.

25. Eksik kalanlar ve OCL'nin amacı

Hoca her türden birer örnek vermeye çalıştığını söyler: değişmez nasıl tanımlanır, implies nedir, birden çok kısıt nasıl birleştirilir, ön koşul nasıl yazılır, saf ve saf olmayan işlemlerde son koşul neyi garanti eder ve türetilen özellikler OCL ile nasıl ifade edilir. Eksik kalanları da sayar:

- 1, 3, 8, 10 ve 11 numaralı gereksinimler hiç incelenmedi; onlar için de OCL yazılması gerekebilir.
- İşlevsel olmayan gereksinimler için hiçbir şey yapılmadı; örneğin kütüphanenin belli bir zamanda belli bir kullanıcı kitlesine aynı anda hizmet vermesiyle ilgili bir kısıt yok.
- Türetilen işlemler (mevcut mu, başkası istemedi mi) için kısıt yazılmadı.
- İade etme ve ceza ödeme gibi başka metotlara bakılmadı.
- Tarih ve para gibi sınıfların var olduğu varsayıldı.
- Parçaların ve ziyaretçilerin nasıl yaratılacağı, kimin yaratacağı ve nasıl bağlanacağı çalışılmadı. Bunların bazıları için OCL yazmak gerekebilir, bazıları için gerekmeyebilir.

Hoca asıl amacı şöyle anlatır: biçimsel tanımlar ve bildirimler yapılır ki tasarım dünyanın herhangi bir yerindeki, UML okumayı bilen birine ya da bir araca verildiğinde, metodun ne yapması ve neyi garanti etmesi gerektiği kısıt altına alınmış olsun: "Sen böyle çalışacaksın, senin tepkin bu olacak, yaptığın işlemler sonuçta bunu garanti edecek. Nasıl yaparsan yap, içine karışmıyorum, ama bunların mutlaka doğru olmasını sağlaman gerekiyor."

Aynı ifadeler bazen değişmez, bazen son koşul olarak farklı şekillerde yazılabilir; tasarımda olduğu gibi OCL yazarken de tek bir doğru yoktur. Analiz yaptıkça yeni gereksinimler (mevcut mu, başkası istemedi mi gibi yeni davranışlar) doğabilir; ceza ve iade işlemleri gibi henüz konuşulmamış açık noktalar ya da hatalar bulunabilir. Bu yüzden müşteriyle tekrar tekrar temasa geçmek gerekebilir.

Hocanın vurgusu: OCL sıkı düşünmeyi sağlar. Bu tür noktaların sürecin erken aşamalarında, tasarım aşamasında, gerçekleştirmeye geçmeden fark edilmesini ve sorun yaratabilecek pürüzlerin en aza indirilmesini amaçlar.

Örnek Sorular

Soru 1 (ekrandan, 00:10:00). Sıralama için "X tam sayı vektörü verildiğinde Y tam sayı vektörü döndürülür ve Y'deki her eleman bir sonrakinden küçük veya eşittir" tanımı yapılmıştır. 3, 2, 1 girdisi için şu çıktılar tanıma uyar mı, uyuyorsa tanıma ne eklenmelidir? (a) 4, 5, 6 (b) 1, 1, 1, 1, 2, 3 (c) 1, 2, 3, 4

Çözüm (derste verildi): Üç çıktı da sıralıdır, bu yüzden ilk tanıma uyar. (a) için çıktının içeriği girdiyle birebir aynı olmalıdır: "Y'deki her eleman X'te de yer almalıdır." (b) bu koşulu sağlar ama sayılar farklıdır; "Y'deki her eleman X'te aynı sayıda yer almalıdır" eklenir. (c) için "yeni eleman yer almamalıdır" da eklenir. Bu üç koşulun hepsi tek seferde "Y, X'in bir permütasyonudur" diye ifade edilir. Python ile denendi: 3, 2, 1'in permütasyonu olan tek sıralı çıktı 1, 2, 3'tür.

Soru 2 (ekrandan, 00:20:00). SQRT fonksiyonu için son koşul ne olabilir?

Çözüm (derste verildi): $Y = SQRT(X)$ demek döngüsel bir tanımdır, çünkü fonksiyon kendisiyle tanımlanır. Girdiyle çıktı arasındaki sayısal ilişki tanımlanmalıdır: $Y \cdot Y = X$. Ön koşul olarak da $X \geq 0$ gerekir. OCL'de: `pre: self >= 0`, `post: self = result * result`.

Soru 3 (derste sözlü, 00:30:09). İki vektör de boşsa `Permütasyon(x, y)` ne döndürür?

Çözüm (derste verildi): Doğru döndürür. Boyut sıfırdır; iki boş vektör birbirinin permütasyonudur. Bu, özyinelemeli tanımın sonlanma durumudur.

Soru 4 (ekrandan, 01:00:00). a ve b isimli Real tipinde iki özelliği olan bir sınıf olsun. Sınıfta a ile b'nin değerlerini takas eden swap işlemi vardır. Son koşulu `post: ___ = ___ and ___ = ___` biçiminde yazın.

Çözüm (derste verildi): `post: a = b@pre and b = a@pre`. OCL'de atama olmadığı için `tmp` gibi geçici bir değişken kullanılmaz; a'nın b'nin ilk hâline, b'nin de a'nın ilk hâline eşit olduğu bildirilir.

Soru 5 (ekrandan, 00:59:05). `context Account::deposit(Real : amount)` için `post: balance = balance + amount` neden yanlıştır? Doğrusu nedir?

Çözüm (derste verildi): OCL bildirimsel bir dildir; `=` atama değil eşitliktir. Bu eşitlik ancak `amount = 0` iken doğru olur. Değer değişiyorsa işlemden önceki değer `@pre` ile gösterilir: `post: balance = balance@pre + amount`. Ön koşul `pre: amount > 0`'dır.

Soru 6 (ekrandan, 01:31:20). Customer, Order, Payment (Credit, Cash, Check) diyagramında ödeme birden fazla bankanın çeki ile yapılsın. Customer nesnesindeyken bu sayı nasıl bulunur?

Çözüm (derste verildi): `self.Order.Check.bankID->size()`. Çoğullama nedeniyle gezintinin sonucu bir koleksiyondur; koleksiyon işlemi `->` ile çağrılır. Payment yerine doğrudan Check'e gidilir, çünkü genelleştirme nedeniyle ödeme bir Check olabilir.

Ders sonrası not: Bu ifade büyük olasılıkla çek sayısını verir. Farklı banka sayısı isteniyorsa tekrarları atmak için büyük olasılıkla `self.Order.Check.bankID->asSet()->size()` yazılmalıdır.

Soru 7 (ekrandan, 01:43:40). "Kitaplar 3 hafta, çok satan kitaplar 2 hafta ödünç alınabilir" gereksinimi için OCL kısıtını yazın. Context ve kısıt tipi ne olmalıdır?

Çözüm (derste verildi): Çok satan, kitabın bir özelliği olduğu için context Kitap'tır. Kesin sayılar olduğu için değişmez yazılır:

```
context Kitap inv: if cokSatan then oduncPeriyodu = 2 else oduncPeriyodu =
3 endif
```

DVD için: `context DVD inv: oduncPeriyodu = 2`.

Soru 8 (ekrandan, 01:52:20). “12 yaş altındaki çocuklar bir kerede en fazla 5 parça ödünç alabilir” kısıtı değişmez mi yoksa ön koşul mu olmalıdır? OCL ile yazın.

> **Çözüm (derste verildi):** Kısıt tüm ziyaretçiler için değil yalnız çocuklar için vardır ve kontrol her ödünç alma işleminde yapılmalıdır; bu yüzden ön koşuldur:
>
> ```
> context Ziyaretci::oduncAl(i: oduncVerilebilirParca) pre: yas <= 12 implies
> oduncAldiklari()->size() < 5
> ```
>
> <= değil < kullanılır; <= 5 olursa beşinci parçadan sonra altıncı da alınabilir.

Soru 9 (derste sözlü, 01:49:50). Ödünç alınan parçaları döndüren `oduncAldiklari()` işleminin dönüş tipi neden küme (Set) seçilmiştir?

> **Çözüm (derste verildi):** Bir başlığın birden fazla kopyası ödünç alınabilir, ama alınan belirli bir kopya aynı anda iki kez alınamaz. Küme aynı elemanın iki kez bulunmamasını garanti eder. İşlem yalnız sorgu yaptığı için ön koşulu yoktur (true).

Dersin Toparlaması

Dersin ilk bölümünde, önceki derste başlanan biçimsel belirtim sıralama problemi üzerinden tamamlandı. “Y sıralıdır” gibi döngüsel ifadeler yerine $\forall i : 1 \leq i \leq n-1,\ y[i] \leq y[i+1]$ yazıldı. Girdiye olan duyarlılık karşı örneklerle gösterildi: sıralı ama yanlış çıktılar ancak girdiyle çıktı arasındaki ilişki (aynı elemanlar, aynı tekrar sayısı, yeni eleman yok) tanımlanınca dışlanır ve bu ilişki özyinelemeli tanımlanan permütasyonla tek seferde verilir. Belirtimin imza, ön koşul ve son koşul olarak üç parçası; sıralama, karekök ve sıralı mı örnekleriyle anlatıldı ve aynı belirtimin OCL karşılığı gösterildi.

İkinci bölümde OCL, UML’in resmî bir parçası, bildirimsel ve güçlü tipli bir kısıt dili olarak tanıtıldı. `context` ile başlayan değişmez, ön koşul ve son koşul kısıtları; `self`, `result`, `@pre`; veri tipleri, anahtar kelimeler, `let`, gezinti, çoğullama ve koleksiyon işlemleri örneklerle ele alındı. Son olarak kütüphane örneğinde ödünç periyodu değişmezleri, ödünç alınanlar sorgusu, çocuklar için ön koşul, yeni ödünç nesnesini ve talep listesini tanımlayan son koşullar ve yaşın türetilmesi yazıldı. Hoca bir sonraki derste davranış modellemesine, yani durumlar, geçişler ve durum diyagramlarına (statechart) geçileceğini söyledi; bu diyagramlardaki geçiş koşulları da OCL ile yazılacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.